

Laboratory / Tutorial

System Overview, Linux, Conversions and Logic Gates

Computer Organization and Digital Foundations

Dr. Yatish Bathla, PhD. (Information Technology), Obuda University, Budapest

Lab Objectives and Schedule

A	**System overview** Inspect CPU, memory, storage and OS	11 min
B	**Linux setup** Start Linux in Docker, learn core commands	13 min
C	**Conversion exercises** Convert by hand, verify with bc, printf, bash	14 min
D	**Logic gate simulation** Build and test gates and small circuits	17 min

60
min
Hands-on

Useful Links

Docker Desktop With PostgreSQL (Windows):

https://www.youtube.com/watch?v=RdPYA-wDhTA

Docker Desktop (Mac):

https://www.youtube.com/watch?v=agkOZr27d3Y

Python (Windows):

https://www.youtube.com/watch?v=ddGTXBhaGWA

Python (Mac):

https://www.youtube.com/watch?v=VJjeMdl6yWA

Golden Gate University | 2026

Prerequisites, Tools and Lab Rules

Hardware

- Laptop or PC
- 4 GB RAM or more
- 10 GB free disk
- Internet access
- Admin rights

Software

- Docker Desktop
- Ubuntu 24.04 container
- nano (inside container)
- CircuitVerse (in browser)

Lab rules

- Save work often
- Understand before you run it
- rm has no undo
- Keep a lab record
- Help, do not copy

A. Looking Inside a Computer

CPU

`lscpu`

Architecture, cores, model, caches

Memory

`free -h`

Total, used and free RAM

Storage

`lsblk | df -h`

Disks, partitions, free space

OS and kernel

`uname -a`

Kernel name, version, architecture

Devices

`lspci | lsusb`

Graphics, network, USB devices

Tip

Inside Docker you see the Docker VM hardware, not the laptop itself

A. Hardware Inspection Commands

```
# CPU: architecture, cores, cache
lscpu
# Memory in human-readable units
free -h
# Disks and partitions
lsblk
# Free space on file systems
df -h
# Kernel and OS details
uname -a
cat /etc/os-release
# Filter one line using a pipe
lscpu | grep -i 'byte order'
```

What to look for

- Architecture and CPU(s)
- Model name and cache sizes
- Total memory
- Byte Order

Remember

- Commands are case-sensitive
- # starts a comment
- | pipes output onward

A. Reading the Output

Output field	Concept	From
`Architecture: x86_64`	Instruction set architecture (ISA)	Lecture 1
`CPU(s), Core(s), Thread(s)`	Parallelism in the organisation	Lecture 1
`L1, L2, L3 cache`	Fast memory between CPU and RAM	Lecture 1
`Byte Order: Little Endian`	Byte ordering in memory	Lecture 2
`Mem: total / available`	Main memory capacity (bytes)	Lecture 1
`lsblk SIZE: 931.5G`	Binary units (GiB) vs marketing GB	Lecture 1

Every line of output is a concept from the lectures made visible.

A. Activity: System Profile Worksheet

#	Item	Command	Your result
1	Architecture	`lscpu`	
2	Number of CPUs	`nproc`	
3	Byte order	`lscpu | grep -i order`	
4	Total RAM	`free -h`	
5	Disk size	`lsblk`	
6	OS name and version	`cat /etc/os-release`	
7	RAM in bytes (compute)	`x 2^30 per GiB`	

B. Why Linux?

1

Open source

Anyone can read, change and share the code

2

Everywhere

Servers, clouds, Android, routers, supercomputers

3

Command line

Direct, scriptable access to the system

4

LTS releases

Ubuntu LTS gets five years of maintenance

Kernel + tools = distribution. This lab uses an Ubuntu 24.04 Docker container.

B. Ways to Get a Linux Environment

Option	Best for	Pros	Cons
Docker container	Any laptop (our choice)	Same on every PC	Terminal only, no desktop
Virtual machine	Any laptop with 8 GB+	Safe, full desktop	Needs RAM, virtualisation
Dual boot	Advanced users	Full speed	Can damage data; back up
Live USB	Quick trial	No install	Changes not kept by default
Cloud terminal	Locked-down PCs	No install	Needs internet; limited

Recommended for this lab: a Docker container (same steps for everyone).

B. Linux in Docker: Step by Step

1

Install Docker

Docker Desktop from docker.com

2

Start Docker

Open Docker Desktop, wait until it says running

3

Open Ubuntu

Run one docker command in a terminal

4

Install tools

apt update, then apt install

```
# On your laptop (PowerShell or Terminal)
docker run -it --name lab1 ubuntu:24.04
# Now inside Ubuntu (prompt shows root@)
apt update
apt install -y procps pciutils bc nano python3
```

Good to know

- exit = leave the container
- docker start -ai lab1 = come back
- You are root: no sudo needed

B. Essential Linux Commands

Command	Purpose
`pwd`	Print current directory
`ls -l`	List files with details
`cd dir`	Change directory
`mkdir dir`	Make a directory
`touch f`	Create an empty file
`cp / mv / rm`	Copy, move, remove
`cat f`	Show file contents
`cmd --help`	Show help for a command
`apt install`	Install software (you are root)

```
# First-session tasks
mkdir lab1
cd lab1
nano notes.txt
cat notes.txt
ls -l
chmod 644 notes.txt
ls -l
ls –help
# docker stop command
docker stop <container name>

# docker start command
docker start <container name>

# docker execute command
docker exec -it <container name> bash
```

C. Converting with Linux Tools

```
# Base N to decimal (bash)
echo $((2#101101))
45
echo $((16#FF))
255
# Decimal to hex / octal
printf '%X\n' 255
FF
printf '%o\n' 255
377
# Decimal to binary (bc)
echo 'obase=2; 45' | bc
101101
```

bc tips

- obase = output base
- ibase = input base
- Hex digits in capitals

Python

- bin(45), hex(255), oct(255)
- int('101101', 2)

C. Exercise Set 1: Convert by Hand

#	Convert	Method	Linux check
1	$101101_2 \rightarrow$ decimal	Weighted sum	`echo $((2#...))`
2	$200_{10} \rightarrow$ binary	Divide by 2	`bc, obase=2`
3	$11111111_2 \rightarrow$ hex and decimal	Group 4 bits	`printf '%X'`
4	$A7_{16} \rightarrow$ binary and decimal	Digit to 4 bits	`echo $((16#A7))`
5	$377_8 \rightarrow$ decimal	Weights 64, 8, 1	`echo $((8#377))`
6	$1100.01_2 \rightarrow$ decimal	Fraction weights	`python3`
7	$0.375_{10} \rightarrow$ binary	Multiply by 2	`bc, obase=2`
8	$1010110_2 \rightarrow$ octal and hex	Group 3 and 4 bits	`printf '%o' '%X'`

C. Exercise Set 2: A Conversion Script

```
#!/bin/bash
n=22
echo "Decimal : $n"
echo "Hex     : $(printf '%X' $n)"
echo "Octal   : $(printf '%o' $n)"
echo "Binary  : $(echo "obase=2; $n" | bc)"
```

```
# Expected output of ./convert.sh 45
Decimal : 22
Hex     : 16
Octal   : 26
Binary  : 10110
```

Your task

- Run for 255, 1000 and 4096
- Record the outputs
- Check one by hand

C. Binary Arithmetic Checks

#	Task	Linux check
1	$1011_2 + 0110_2$	`echo $((2#1011 + 2#0110))`
2	$1101_2 - 0110_2$ using 2's complement (4 bits)	`echo $((2#1101 - 2#0110))`
3	8-bit 2's complement of 00000110_2	`printf '%X\n' $(( (~2#110 + 1) & 0xFF ))`
4	Range of an 8-bit signed number	`by formula: -2^7 to 2^7 - 1`
5	Does 0111 + 0101 overflow in 4 bits?	`Check the sign of the result`

Hand method first; the command is only the check.

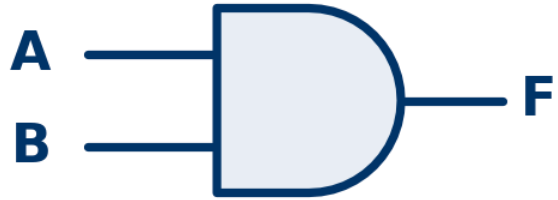
D. Setting Up the Logic Simulator

1. **Get the tool**
   Open CircuitVerse in your browser. Nothing to install

2. **Not in Docker**
   The simulator runs on your laptop, not in the container

3. **Place parts**
   Two input pins, one AND gate, one output pin

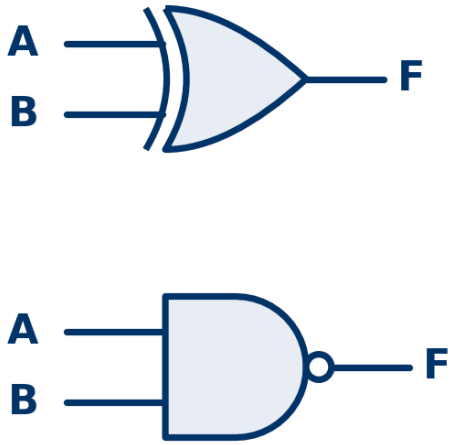
4. **Wire and test**
   Use the Poke tool to toggle the inputs



Poke tool
Click an input pin to toggle 0 and 1 and watch the output

D. Lab 1: Gates and Truth Tables

A B	AND	OR	NAND	NOR	XOR
0 0					
0 1					
1 0					
1 1					



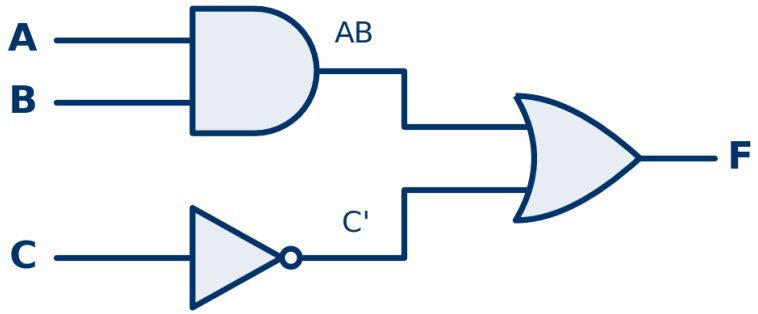
Build
One circuit, six gates, labelled pins

Test
Poke each input combination

Record
Screenshot and completed table

D. Lab 2: Circuits and Half Adder

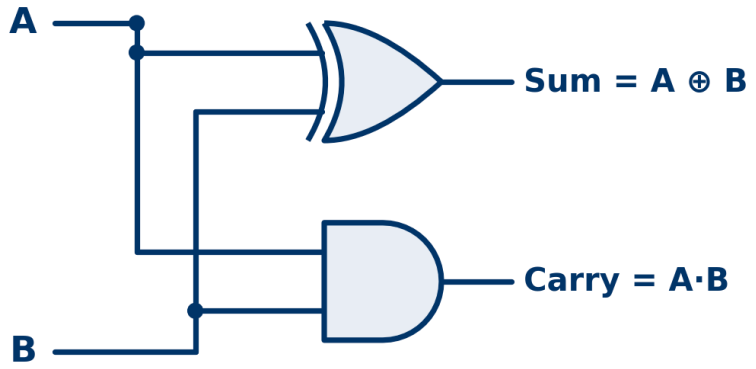
Part 1: F = A·B + C'



Test

- All 8 rows of A B C
- F should be 1 for 000, 010, 100, 110, 111

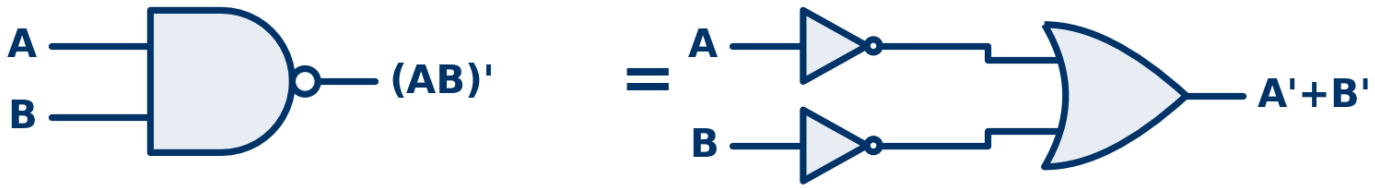
Part 2: Half adder



Test

- All 4 rows of A B
- 1 + 1 gives Sum 0 and Carry 1 (binary 10)

D. Lab 3: De Morgan and NAND



Part 1: both outputs must match for every input

Part 1: De Morgan

- Build (AB)' and A' + B'
- Compare 4 rows
- Outputs must match

Part 2: NAND only

- Build NOT, AND and OR from NAND gates
- Test each against the real gate

Extension: full adder

- Two half adders + an OR for carry out
- Test all 8 input rows

Wrap-Up, Submission and Assessment

Submission checklist

- System profile with lscpu screenshot
- Conversion tables and convert.sh output
- Binary arithmetic checks
- Screenshots of three circuits and truth tables
- Short reflection (3 sentences)

Component	Weight
System overview	20%
Conversions and script	30%
Logic simulation	40%
Presentation, completeness	10%

Next unit

Simplifying Boolean functions and combinational circuits